

BTG Pactual Credit Engine — Backend Challenge

Build a REST API that classifies customers into risk clusters, calculates personalised credit limits, and estimates monthly income. The classification engine must be data-driven — the business rules defined below should be representable as configuration, not buried in logic.

 AI is welcome here. We want to know *how* you used it, not *if* you did.

Input Data

Customer Schema

Field	Type	Description
id	string	Unique identifier
name	string	Full name
age	integer	Age in years
score	integer	Credit bureau score (0–1000)
has_market_debt	boolean	Whether the customer has any recorded market debt
market_debt_types	string[]	Active debt types (see below)
location.city	string	City of residence
location.state	string	State abbreviation (e.g. SP , RJ)
location.region	string	Norte · Nordeste · Centro-Oeste · Sudeste · Sul
job_title	string	Free-text job title

Valid `market_debt_types` values:

Value	Meaning
<code>credit_card</code>	Active credit card (non-defaulted)
<code>personal_loan</code>	Active personal loan (non-defaulted)
<code>mortgage</code>	Mortgage (non-defaulted)
<code>credit_default</code>	Credit card in collections ⚠
<code>loan_default</code>	Loan in collections ⚠

Business Rules

Design note: The tables in this section define what the system must enforce. How you represent and load these rules is your design decision — be prepared to justify it.

1. Customer Clusters

Evaluate in priority order. Assign the **first** cluster whose conditions are fully met.

Priority	Cluster ID	Name	Score	Age	Debt Condition	Base Limit	Cap
1	CLUSTER_A	Diamond	≥ 700	25–60	<code>has_market_debt == false</code>	R\$ 50,000	R\$ 100,000
2	CLUSTER_B	Gold	≥ 500	18–65	<code>no credit_default or loan_default in market_debt_types</code>	R\$ 20,000	R\$ 40,000
3	CLUSTER_C	Silver	≥ 300	—	—	R\$ 5,000	R\$ 10,000

Priority	Cluster ID	Name	Score	Age	Debt Condition	Base Limit	Cap
4	CLUSTER_D	Bronze	—	—	catch-all	R\$ 0	R\$ 0 — denied

2. Job Title Categories

Match case-insensitively anywhere in the job title. Evaluate top-down; first match wins.

Priority	Category	Multiplier	Keywords
1	EXECUTIVE	×2.0	CEO, CFO, CTO, COO, CIO, CMO, Chief, President, Vice President, VP, Director
2	SENIOR_PROFESSIONAL	×1.5	Senior, Lead, Manager, Coordinator, Supervisor, Principal
3	MID_PROFESSIONAL	×1.0	Engineer, Analyst, Developer, Specialist, Designer, Accountant, Consultant, Architect
4	JUNIOR_PROFESSIONAL	×0.7	Junior, Trainee, Intern, Apprentice, Assistant, Associate
5	OTHER	×0.8	<i>(no keyword matched)</i>

3. Monthly Income

Derived from cluster assignment and job category (values in BRL):

	EXECUTIVE	SENIOR_PROFESSIONAL	MID_PROFESSIONAL	JUNIOR_PROFESSIONAL	OTHER
CLUSTER_A	30,000	20,000	12,000	8,000	10,000
CLUSTER_B	20,000	15,000	8,000	5,000	6,500
CLUSTER_C	10,000	7,000	5,000	3,000	4,000
CLUSTER_D	0	0	0	0	0

4. Credit Limit Formula

```
approved_limit = round_to_nearest_100(  
    min( base_limit × job_multiplier × penalty_factor, cluster_cap )  
)
```

CLUSTER_D always yields `approved_limit = 0`.

5. Penalty Rules

Applied after the job multiplier, before the cluster cap.

Priority	Rule ID	Trigger	Effect
1	DEFAULT_DEBT_PENALTY	credit_default OR loan_default in market_debt_types	×0.5

API

POST /customers/classify

Accept a customer object in the request body, classify it, and return the enriched record. The application is stateless — nothing is persisted between calls.

Request body: Customer object (schema defined above).

Response: The same object enriched with all calculated fields from the output contract.

Testing (Required)

Both unit and integration tests are required.

Unit tests must cover the core classification logic in isolation:

- Cluster assignment for each cluster, including boundary conditions (e.g. score exactly at threshold)
- Job category matching, including case-insensitivity and priority ordering
- Credit limit calculation: base formula, penalty application, cap enforcement, and `round_to_nearest_100`
- Monthly income lookup for all cluster × job category combinations
- CLUSTER_D denial (approved = false, approved_limit = 0)

Integration tests must exercise the full request/response cycle:

- `POST /customers/classify` with valid input returns correct output contract
- `POST /customers/classify` with invalid or missing fields returns appropriate error responses

No expected output file (e.g., JSON fixtures with precomputed responses) is required or expected to be submitted. Instead, the implementation and tests must derive results from the business rules defined in this specification. We expect:

The classification logic to be fully implemented, not hard-coded Test cases to assert outcomes produced by the implementation itself Clear evidence, through test coverage and/or test naming, of the reasoning behind each expected outcome

Your test suite must be runnable with a single command (e.g. `pytest` , `dotnet test` , `npm test`).

Evaluation

Area	What we look for
Problem Solving	Correct rule implementation, edge-case handling
Architecture	Separation of concerns, data-driven design

Area	What we look for
Code Quality	Readability, error handling, input validation
Testing	Coverage of classification paths and edge cases
Documentation	API docs, architecture decisions
AI Collaboration	How you used AI, critical thinking about AI output

AI Journey (Required)

AI usage is expected and encouraged — we want to see how you think with it, not whether you avoided it. Include an `ai-journey/` folder in your repository documenting your process:

```
ai-journey/  
├─ README.md      # Brief summary: which tools you used and for what  
├─ prompts.md     # Key prompts, what they produced, and how you iterated  
└─ learnings.md   # What worked, what didn't, what you'd do differently
```

`prompts.md` — focus on the interesting interactions, not a full log:

Prompt: Cluster assignment logic

****Tool:**** GitHub Copilot / ChatGPT-4 / etc.

****What I asked:****

"Given a priority-ordered list of cluster conditions, write a function that evaluates each cluster in order and returns the first match."

****What happened:****

The initial suggestion used nested if/else which became hard to maintain.
I asked for a data-driven approach instead and got a cleaner result.

****Final solution:****

[brief description or code snippet]

learnings.md — honest reflection on the experience:

What worked well

- AI was fast for generating test data and boilerplate
- Helped explore regex patterns for job-title keyword matching

What didn't work

- Rounding logic was subtly wrong on the first attempt – had to verify manually
- AI missed the priority-order constraint in cluster evaluation

What I'd do differently

- Prompt for edge cases explicitly before accepting a solution
- Use AI earlier in the design phase, not just for implementation

The depth and honesty of this documentation matters as much as the code itself.